Kịch Bản Thuyết Trình Chi Tiết

XLA Face Privacy System

Môn Xử Lý Ảnh

Vai trò	Thành viên	Phụ trách
Leader / Security	**Đạt**	AES-256-GCM · PBKDF2 · Blur Engine
Backend / API	Khánh	FastAPI · Pipeline · Streaming
YOLO / Registry	Khoa	YOLOv11 · InsightFace · FaceDB

Thời lượng: ≈ 30 phút
Ngày: Ngày 15 tháng 3 năm 2026
Repository: `github.com/Poicitaco/XLA_T3`
Branch: `feature/cipher`

LƯU Ý

Chuẩn bị trước khi trình bày:

- Server đang chạy: `uvicorn src.app_api:app -host 0.0.0.0 -port 8000`
- Admin password đã được setup (POST /admin/setup)
- Có ít nhất 2 clip `.enc` trong `data/clips/secure/`
- VS Code mở sẵn: `src/clip_recorder.py`, `src/admin_auth.py`, `src/privacy.py`, `src/face_recognition.py`
- Browser: 3 tab sẵn sàng tại `localhost:8000`
- Font size VS Code $\geq$ 16, zoom browser 110%

Mục lục

1 Tổng Quan Kịch Bản 0:00 - 30:00 **2**

2 Phần 1 — Đạt: Mở Đầu và Kiến Trúc 0:00 - 6:00 **2**
2.1 Đặt vấn đề 0:00 - 1:30 2
2.2 Giới thiệu hai tầng bảo vệ 1:30 - 3:00 2
2.3 Kiến trúc tổng thể 3:00 - 5:00 3
2.4 Chuyển sang Khoa 5:30 - 6:00 3

3 Phần 2 — Khoa: YOLO + InsightFace + Face Registry 6:00 - 12:00 **4**
3.1 YOLOv11 Face Detection 6:00 - 8:00 4
3.2 InsightFace Embedding 8:00 - 10:00 5
3.3 Face Registry Demo 10:00 - 12:00 6

4 Phần 3 — Khánh: Backend API + Live Pipeline 12:00 - 18:00 **7**
4.1 FastAPI Architecture 12:00 - 14:00 7
4.2 Live Pipeline Architecture 14:00 - 16:00 8
4.3 Hot-reload Config 16:00 - 17:30 9
4.4 Chuyển sang Đạt 17:30 - 18:00 9

5 Phần 4 — Đạt: Security Layer 18:00 - 25:00 **10**
5.1 Tại sao cần mã hóa? 18:00 - 19:30 10
5.2 AES-256-GCM Encryption 19:30 - 21:30 10
5.3 PBKDF2-SHA256 Key Derivation 21:30 - 23:00 11
5.4 Constant-Time Comparison 23:00 - 23:45 12
5.5 Blur Engine — 7 Privacy Modes 23:45 - 24:30 13
5.6 Demo Decrypt trên Secure Archive 24:30 - 25:00 14

6 Phần 5 — Demo End-to-End + Q&A 25:00 - 30:00 **14**
6.1 Demo end-to-end hoàn chỉnh 25:00 - 27:30 14
6.2 Privacy Modes Showcase 27:30 - 29:00 16
6.3 Kết luận 29:00 - 30:00 16

7 Câu Hỏi Thường Gặp — Đáp Án Chuẩn Bị Sẵn **16**

8 Checklist Kỹ Thuật Trước Buổi Trình Bày **18**

1. Tổng Quan Kịch Bản

0:00 - 30:00

Phần	Người trình bày	Nội dung	Thời gian
1	Đạt	Mở đầu, vấn đề, kiến trúc tổng thể	0:00–6:00
2	Khoa	YOLO Detection + InsightFace + Face Registry	6:00–12:00
3	Khánh	FastAPI Backend + Live Pipeline + Streaming	12:00–18:00
4	Đạt	Security Layer (Blur/Encrypt/Decrypt)	18:00–25:00
5	**Cả nhóm**	Demo end-to-end + Q&A	25:00–30:00

2. Phần 1 — Đạt: Mở Đầu và Kiến Trúc

0:00 - 6:00

2.1 Đặt vấn đề

0:00 - 1:30

[ĐẠT] Lời mở đầu

"Xin chào thầy cô và các bạn. Em là Đạt, đại diện nhóm chúng em trình bày đồ án môn Xử Lý Ảnh với đề tài: **Hệ thống camera thông minh bảo vệ quyền riêng tư khuôn mặt theo thời gian thực.**"

"Câu hỏi đặt ra: Khi camera an ninh quay mọi người đi qua — khách, người giúp việc, hàng xóm — họ có đồng ý bị nhận dạng và lưu trữ khuôn mặt không? Theo luật GDPR và quy định bảo vệ dữ liệu Việt Nam, câu trả lời là **không**. Nhưng nếu xóa hết camera thì ta lại mất bằng chứng khi có sự cố."

"Hệ thống của nhóm em giải quyết đúng mâu thuẫn này: **bảo vệ quyền riêng tư theo mặc định, đồng thời không mất đi bằng chứng.**"

LƯU Ý

Nói chắc, đứng thẳng, nhìn vào mắt thầy cô khi nói câu định nghĩa cuối. Đây là câu "hook" quan trọng nhất trong phần mở đầu.

2.2 Giới thiệu hai tầng bảo vệ

1:30 - 3:00

THAO TÁC

Chuyển sang slide **"Dual Storage Architecture"**. Dùng bút laser hoặc tay chỉ vào từng tầng khi giải thích.

[ĐẠT] Hai tầng bảo vệ

"Hệ thống hoạt động theo nguyên lý **Dual Storage**. Mỗi khi camera phát hiện khuôn mặt, hệ thống ghi lại **đồng thời hai bản**:

Tầng 1 — Blurred Stream: Video khuôn mặt đã bị che giấu, lưu định dạng MP4 thông thường. Bất kỳ ai trong hệ thống đều có thể xem — không phân biệt được danh tính người trong clip.

Tầng 2 — Encrypted Store: Video gốc hoàn toàn nguyên vẹn, được mã hóa bằng AES-256-GCM và lưu dưới dạng file `.enc`. *Chỉ* người quản trị biết mật khẩu mới giải mã được. Dùng khi cần điều tra pháp lý."

2.3 Kiến trúc tổng thể

3:00 - 5:00

THAO TÁC

Mở VS Code, trỏ vào cây thư mục bên trái theo từng dòng. Đọc to tên file và giải thích ngắn.

[ĐẠT] Kiến trúc 3 tầng

"Kiến trúc hệ thống gồm 3 tầng:"

"**Tầng Detection** — `src/detector.py`, `src/face_recognition.py`: Khoa phụ trách. YOLOv11 phát hiện khuôn mặt, InsightFace trích xuất embedding để nhận diện thành viên gia đình."

"**Tầng Backend** — `src/app_api.py`, `src/backend_api.py`: Khánh phụ trách. FastAPI xử lý real-time pipeline, phân luồng video đến đúng module, phục vụ giao diện frontend qua REST và WebSocket."

"**Tầng Security** — `src/privacy.py`, `src/clip_recorder.py`, `src/admin_auth.py`: Em phụ trách. Blur engine xử lý 7 chế độ che giấu, mã hóa AES-256-GCM với key derive từ PBKDF2-SHA256."

THAO TÁC

Show nhanh cấu trúc thư mục trong terminal:

```
Get-ChildItem src/ | Select-Object Name
```

2.4 Chuyển sang Khoa

5:30 - 6:00

[ĐẠT] Chuyển phần

"Bây giờ em mời **Khoa** trình bày về phần nền tảng của pipeline — hệ thống phát hiện và nhận diện khuôn mặt. Khoa ơi."

3. Phần 2 — Khoa: YOLO + InsightFace + Face Registry 6:00 - 12:00

3.1 YOLOv11 Face Detection 6:00 - 8:00

THAO TÁC

Mở file `src/detector.py`. Scroll đến class `FaceDetector`, highlight phần `__init__` và method `detect_faces`.

[KHOA] YOLOv11 Detector

"Em dùng **YOLOv11n-face** — phiên bản nano của YOLO thế hệ 11, được fine-tune đặc biệt để phát hiện khuôn mặt. Model weights nằm tại `models/yolov11n-face.pt`."

THAO TÁC

Chỉ vào file `src/detector.py`, highlight đoạn khởi tạo:

Listing 1: src/detector.py — Khởi tạo YOLO

```python
class FaceDetector:
    def __init__(
        self,
        model_path: str = "models/yolov11n-face.pt",
        conf_threshold: float = 0.35,    # Nguong tin cay
        iou_threshold:  float = 0.50,    # Non-Maximum Suppression
        imgsz:          int   = 640,     # Kich thuoc anh inference
    ) -> None:
        yolo_cls = self._resolve_yolo_class()
        self.model = yolo_cls(model_path)
```

[KHOA] Giải thích tham số

"Ba tham số quan trọng:
conf_threshold = 0.35: Chỉ chấp nhận detection có độ tin cậy trên 35%. Nếu để thấp quá sẽ có nhiều false positive — nhầm đồ vật thành mặt người.
iou_threshold = 0.50: Ngưỡng IoU cho Non-Maximum Suppression. Khi hai bounding box chồng nhau quá 50% thì giữ lại box có confidence cao hơn.
imgsz = 640: YOLO resize frame về 640×640 trước khi inference để đảm bảo hiệu suất nhất quán bất kể độ phân giải camera."

THAO TÁC

Scroll xuống method `detect_faces`, highlight vòng lặp kết quả:

Listing 2: src/detector.py — detect_faces

```python
def detect_faces(self, frame: np.ndarray) -> List[List[int]]:
    results = self.model(
        frame,
```

```
        imgsz=self.imgsz,
        conf=self.conf_threshold,
        iou=self.iou_threshold,
        verbose=False,
    )
    boxes = []
    for r in results:
        for box in r.boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            # Chuyen tu [x1,y1,x2,y2] sang [x,y,w,h]
            boxes.append([x1, y1, x2 - x1, y2 - y1])
    return boxes
```

[KHOA] Format bounding box

"YOLO trả về tọa độ góc trên trái và góc dưới phải `[x1,y1,x2,y2]`. Em convert sang định dạng `[x, y, width, height]` — định dạng phổ biến hơn khi vẽ rectangle với OpenCV."

3.2 InsightFace Embedding

8:00 - 10:00

THAO TÁC

Mở file `src/face_recognition.py`. Highlight class `FaceRecognizer`.

Listing 3: src/face_recognition.py — InsightFace setup

```
from insightface.app import FaceAnalysis

class FaceRecognizer:
    def __init__(
        self,
        model: str = "buffalo_l",        # Model buffalo_l: 512-dim
            embedding
        det_size: tuple = (256, 256),    # Kich thuoc detect noi bo
            InsightFace
    ) -> None:
        self.app = FaceAnalysis(
            name=model,
            providers=["CPUExecutionProvider"]  # Chay tren CPU
        )
        self.app.prepare(ctx_id=0, det_size=det_size)
```

[KHOA] InsightFace Buffalo-L

"InsightFace **buffalo_l** là model nhận diện khuôn mặt state-of-the-art, tạo ra vector embedding **512 chiều** — gọi là "vân tay số" của khuôn mặt.
Mỗi người có một vector embedding đặc trưng. Để nhận diện, em tính **cosine similarity** giữa embedding đang quan sát và embedding đã đăng ký trong database:"

Listing 4: src/face_recognition.py — Cosine Similarity

```python
@staticmethod
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    a_norm = np.linalg.norm(a)
    b_norm = np.linalg.norm(b)
    if a_norm == 0.0 or b_norm == 0.0:
        return -1.0
    # Gia tri tu -1 (nguoc hoan toan) den +1 (giong hoan toan)
    return float(np.dot(a, b) / (a_norm * b_norm))
```

[KHOA] Ngưỡng nhận diện

"Giá trị similarity từ **-1 đến +1**. Ngưỡng mặc định là **0.35**: nếu similarity > 0.35, đây là thành viên đã đăng ký $\rightarrow$ *không blur*. Nếu similarity $\leq 0.35 \rightarrow$ người lạ $\rightarrow$ *bị blur*."

LƯU Ý

Minh hoạ trực quan: "Nếu trong database em có vector của Khoa, và camera nhìn thấy Khoa, similarity sẽ khoảng 0.85. Một người lạ chưa đăng ký sẽ cho similarity âm hoặc gần 0."

3.3 Face Registry Demo

10:00 - 12:00

THAO TÁC

Mở browser, chuyển đến tab `http://localhost:8000/admin/face-registry.html`. Chỉ vào từng thành phần của UI.

[KHOA] Demo Face Registry

"Đây là trang đăng ký thành viên. Quy trình đăng ký rất đơn giản: nhập tên thành viên, upload ảnh khuôn mặt. Backend tự động trích xuất embedding và lưu vào database."

THAO TÁC

Mở VS Code, trỏ vào thư mục `data/members/`:

```powershell
Get-ChildItem data/members/ -Recurse |
  Where-Object { -not $_.PSIsContainer } |
  Group-Object DirectoryName |
  Select-Object Name, Count
```

[KHOA] Giải thích dữ liệu

"Hiện tại database có 2 thành viên: **Đạt** và **Khoa**, mỗi người **100 ảnh mẫu**. Nhiều ảnh mẫu từ nhiều góc độ và điều kiện ánh sáng khác nhau giúp embedding ổn định hơn và giảm thiểu nhận diện sai.
File JSON lưu embedding được persist tại `data/family_members.json`."

THAO TÁC

Show nhanh file `data/family_members.json`, highlight cấu trúc: `member_id`, `name`, `threshold`, `embeddings` (danh sách vector).

[KHOA] Chuyển phần

"Em xin bàn giao cho Khánh trình bày về Backend API và hệ thống pipeline thời gian thực."

4. Phần 3 — Khánh: Backend API + Live Pipeline 12:00 - 18:00

4.1 FastAPI Architecture 12:00 - 14:00

THAO TÁC

Mở file `src/app_api.py`. Scroll lên đầu file, highlight phần docstring liệt kê tất cả endpoint.

[KHÁNH] Giới thiệu FastAPI

"Backend em xây dựng bằng **FastAPI** — Python async web framework, tự động generate OpenAPI documentation.
Toàn bộ API được chia thành nhóm chức năng rõ ràng:"

Listing 5: src/app_api.py — Endpoint catalog

```
# Health / Info
GET  /health           # Kiem tra server song
GET  /info             # Version, capabilities, mode list
GET  /presets          # 4 preset: fast/balanced/strong/strict

# Xu ly anh tinh (stateless)
POST /anonymize        # Cha mat trong 1 anh upload
POST /lock             # Ma hoa AES-GCM tung vung mat
POST /unlock           # Giai ma, phuc hoi mat goc
POST /recognize        # Nhan dien tat ca mat trong anh

# Camera va Pipeline
GET  /cameras          # Liet ke camera kha dung
POST /pipeline/start   # Bat pipeline camera
DELETE /pipeline/stop  # Tat pipeline
GET  /pipeline/status  # Trang thai + thong ke live
PATCH /pipeline/config# Cap nhat config khong can restart

# Streaming
GET  /stream/mjpeg     # MJPEG stream (dung cho <img src="">)
GET  /stream/sse       # Server-Sent Events
WS   /stream/ws        # WebSocket (?format=binary|json)

# Clip management
GET  /clips            # Danh sach clip da luu
```

```
POST /clips/decrypt    # Giai ma clip (admin only)
DELETE /clips/{id}     # Xoa mot clip
```

[KHÁNH] CORS và Security

"Một điểm quan trọng trong cấu hình backend:"

Listing 6: src/app_api.py — CORS middleware

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],      # Dev mode; production nen gioi han
    allow_methods=["*"],
    allow_headers=["*"],
)
```

4.2 Live Pipeline Architecture

14:00 - 16:00

THAO TÁC

Mở browser, chuyển đến `http://localhost:8000/admin/dashboard.html`. Chỉ cho thầy cô xem layout tổng thể trước khi start pipeline.

[KHÁNH] Giải thích pipeline

"Khi nhấn Start, backend khởi động một background thread. Luồng xử lý như sau:"

Listing 7: Pipeline flow (minh họa)

```
# 1. Mo camera
cap = cv2.VideoCapture(camera_id)

# 2. FrameReader: doc frame tu camera theo thread rieng
# -> giam delay so voi doc dong bo

# 3. Moi frame:
if frame_index % detect_every == 0:
    boxes = detector.detect_faces(frame)    # YOLO
    boxes = smoother.smooth(boxes)          # EMA smoothing

# 4. RecognitionWorker: nhan dien async
#    So sanh embedding voi FamilyDatabase
#    -> identity: "Dat" | "Khoa" | None (nguoi la)

# 5. anonymize_faces():
#    Nhung mat co identity=None -> BLUR
#    Nhung mat co identity != None -> giu nguyen + ve ten

# 6. ClipRecorder.push_frame():
#    raw frame  -> .enc (AES-256-GCM)
#    display frame -> .mp4 (blurred)
```

THAO TÁC

Thao tác live: chọn camera từ dropdown, mode **headcloak**, click **Start Pipeline**. Chờ stream hiện lên, chỉ cho thầy cô thấy: (a) người đã đăng ký không bị blur, (b) người lạ bị che hoàn toàn.

[KHÁNH] Bbox Smoother

"Một chi tiết kỹ thuật nhỏ nhưng quan trọng: chúng em có module `bbox_smoother.py` dùng **Exponential Moving Average** để làm mịn bounding box qua các frame.
Không có smoother, bounding box sẽ nhảy lung tung frame-by-frame. Với EMA alpha = 0.7, box di chuyển mượt mà và ổn định hơn nhiều."

4.3 Hot-reload Config

16:00 - 17:30

THAO TÁC

Trong khi pipeline đang chạy, dùng UI đổi mode từ `headcloak` sang `pixelate`. Chỉ cho thầy cô thấy thay đổi ngay lập tức trên stream.

Listing 8: src/app_api.py — PATCH /pipeline/config

```python
@app.patch("/pipeline/config")
async def update_config(req: ConfigUpdateRequest):
    """Cap nhat config pipeline KHONG can restart server."""
    global _pipeline_session
    if _pipeline_session is None:
        raise HTTPException(404, "Pipeline not running")

    # Atomic update: thay the config object in-place
    new_cfg = dc_replace(_pipeline_session.config, **req.dict(...))
    _pipeline_session.config = new_cfg
    return {"ok": True, "config": asdict(new_cfg)}
```

[KHÁNH] Hot-reload

"Endpoint PATCH `/pipeline/config` cho phép thay đổi toàn bộ tham số của pipeline *mà không cần tắt server*. Đây là tính năng production-ready quan trọng — trong thực tế admin không thể restart camera mỗi lần cần đổi mode."

4.4 Chuyển sang Đạt

17:30 - 18:00

[KHÁNH] Chuyển phần

"Em xin bàn giao lại cho Đạt để trình bày phần quan trọng nhất của hệ thống — tầng bảo mật, mã hóa và decryption. Đạt ơi."

5. Phần 4 — Đạt: Security Layer 18:00 - 25:00

LƯU Ý

Đây là phần **dài và quan trọng nhất** của toàn bài. Đạt cần trình bày tự tin, chắc chắn, và giải thích đủ sâu để thầy cô thấy rõ kiến thức bảo mật thực sự. Đừng đọc code — giải thích ý nghĩa từng dòng.

5.1 Tại sao cần mã hóa? 18:00 - 19:30

THAO TÁC

Quay lại slide **"Dual Storage — Why Encryption?"**.

[ĐẠT] Động lực mã hóa

"Khánh vừa demo pipeline blur rất đẹp. Nhưng nếu chỉ dừng ở blur thì hệ thống có một lỗ hổng nghiêm trọng: **bằng chứng bị mất**.
Giả sử tối nay có trộm đột nhập. Camera ghi lại clip. Nhưng clip đã blur hoàn toàn — khuôn mặt kẻ trộm không còn. Video đó *vô dụng* cho điều tra.
Giải pháp: lưu song song hai bản — bản public đã blur để mọi người xem bình thường, và bản gốc mã hóa chỉ admin mới giải mã được.
Đây là triết lý thiết kế: **Privacy by Default, Evidence on Demand.**"

5.2 AES-256-GCM Encryption 19:30 - 21:30

THAO TÁC

Mở file `src/clip_recorder.py`. Scroll đến hàm `encrypt_clip` (khoảng dòng 110). Highlight toàn bộ hàm.

Listing 9: src/clip_recorder.py — encrypt_clip

```python
def encrypt_clip(
    frames: List[np.ndarray],
    fps: float,
    password: str
) -> bytes:
    """
    Ma hoa frames bang AES-256-GCM.
    Output: [ 32B salt ][ 12B nonce ][ AES-GCM ciphertext + tag ]
    """
    # Buoc 1: Serialize frames thanh JPEG stream
    plaintext = _pack_frames(frames, fps)

    # Buoc 2: Sinh ngau nhien, moi clip mot salt RIENG
    salt  = secrets.token_bytes(32)    # 256-bit salt
    nonce = secrets.token_bytes(12)    # 96-bit nonce cho GCM

```

```
    # Buoc 3: Derive key tu password + salt qua PBKDF2
    key = derive_clip_key(password, salt)  # -> 32 bytes (AES-256)

    # Buoc 4: Ma hoa AES-256-GCM
    ct = AESGCM(key).encrypt(nonce, plaintext, None)

    # Buoc 5: Ghi file: salt | nonce | ciphertext
    return salt + nonce + ct
```

[ĐẠT] Giải thích từng bước

"**Bước 1 — Serialize:** Frames được nén thành JPEG quality 90 rồi đóng gói theo format length-prefixed: `[4B số frame][4B fps][4B len][jpeg bytes]...` Tại sao JPEG 90? Đủ chất lượng để nhận diện pháp lý, file nhỏ hơn raw.

Bước 2 — Random salt per clip: Điểm cực kỳ quan trọng: *mỗi clip có một salt 32-byte ngẫu nhiên riêng.* Dù admin dùng cùng một password, key của clip A và clip B **hoàn toàn khác nhau**. Hacker lấy được key của một clip cũng không dùng được cho clip khác.

Bước 3 — PBKDF2 Key Derivation: Password → `PBKDF2-SHA256(password, salt, 390000 iterations)` → 32-byte key. Tôi sẽ giải thích chi tiết phần này ngay sau.

Bước 4 — AES-256-GCM: Tại sao *GCM* chứ không phải CBC hay CTR? GCM là **Authenticated Encryption** — ngoài mã hóa, còn tạo ra Authentication Tag để đảm bảo toàn vẹn. Nếu ai sửa nội dung file `.enc` dù chỉ 1 bit, decrypt sẽ fail ngay — không trả về dữ liệu sai, điều cực kỳ quan trọng với bằng chứng pháp lý.

Bước 5 — File format: File `.enc` = `salt(32B)` + `nonce(12B)` + `ciphertext`. Không có overhead gì — compact và self-contained."

5.3 PBKDF2-SHA256 Key Derivation

21:30 - 23:00

THAO TÁC

Mở file `src/admin_auth.py`. Scroll đến đầu file, highlight hằng số `_ITERATIONS` và hàm `_pbkdf2`.

Listing 10: src/admin_auth.py — PBKDF2

```
# OWASP 2023 minimum for PBKDF2-SHA256
_ITERATIONS = 390_000
_KEY_LEN    = 32    # 256-bit AES key

def _pbkdf2(password: str, salt: bytes,
            iterations: int = _ITERATIONS) -> bytes:
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=_KEY_LEN,       # Output: 32 bytes
        salt=salt,
        iterations=iterations,
    )
    return kdf.derive(password.encode("utf-8"))
```

[ĐẠT] Giải thích PBKDF2

"**PBKDF2** là *Password-Based Key Derivation Function 2*. Nó biến một password ngắn, dễ nhớ thành một key mật mã học đủ mạnh.
Con số **390.000 iterations** không phải chọn ngẫu nhiên — đây là **chuẩn OWASP 2023** cho PBKDF2-SHA256.
Ý nghĩa thực tế: Để thử một password, máy tính phải tính SHA256 *390.000 lần*. Với một máy hiện đại chạy 1 triệu hash/giây, brute-force 8 ký tự mixed-case + số + symbol tốn khoảng **hàng trăm ngàn năm**.
Và password admin **không bao giờ lưu dưới dạng plaintext**. Nhìn vào file credentials:"

THAO TÁC

Mở `data/admin_credentials.json`, chỉ cho thầy cô xem.

Listing 11: data/admin_credentials.json

```
{
  "salt": "base64-encoded-32-bytes...",
  "hash": "base64-encoded-pbkdf2-hash...",
  "iterations": 390000,
  "created_at": 1741234567.89
}
```

[ĐẠT] Credentials file

"File này chỉ chứa **salt** và **hash**. Không có password, không có key, không có gì có thể reverse. Ngay cả em — người viết code hệ thống — cũng *không biết* password admin là gì nếu không được admin cung cấp trực tiếp."

5.4 Constant-Time Comparison

23:00 - 23:45

THAO TÁC

Trong file `src/admin_auth.py`, highlight hàm `_ct_equal`.

Listing 12: src/admin_auth.py — Timing Attack Prevention

```python
def _ct_equal(a: bytes, b: bytes) -> bool:
    """
    Constant-time comparison - ngan Timing Attack.
    Python '==' dung lai ngay khi gap byte khac.
    hmac.compare_digest LUON so sanh het ca chuoi.
    """
    return hmac.compare_digest(a, b)
```

[ĐẠT] Timing Attack

"Đây là một detail security nhỏ nhưng thể hiện tư duy bảo mật chuyên sâu.
Nếu dùng `a == b` bình thường, Python so sánh từng byte và dừng lại ngay khi gặp byte

khác nhau. Hacker có thể đo thời gian response của server: - Nếu response nhanh → sai từ byte đầu tiên - Nếu response chậm hơn chút xíu → vài byte đầu đúng rồi
Qua hàng nghìn lần thử, hacker có thể đoán dần password đúng từng byte. Đây gọi là **Timing Attack**.
`hmac.compare_digest` luôn chạy một thời gian cố định bất kể input khác nhau ở đâu → loại bỏ hoàn toàn timing leak."

5.5 Blur Engine — 7 Privacy Modes

23:45 - 24:30

THAO TÁC

Mở file `src/privacy.py`. Scroll nhanh qua các hàm `_fast_gaussian_blur`, `_pixelate`, `_head_cloak`, `_obliterate_features` để thầy cô thấy sự đa dạng.

[ĐẠT] 7 Privacy Modes

"Hệ thống hỗ trợ **7 chế độ ẩn danh** từ nhẹ đến cực mạnh:"

7 Privacy Modes

Mode	Kỹ thuật	Mô tả	Tốc độ
`blur`	Gaussian Blur	Làm mờ Gaussian downscale 20%	Nhanh
`pixelate`	Bilinear + Nearest	Pixel hóa mosaic 16px block	Nhanh
`solid`	Constant fill	Hình chữ nhật màu đặc #282828	Rất nhanh
`headcloak`	Mix + Noise + Blur	Che đầu + thêm noise ngẫu nhiên	Trung bình
`neckup`	Expand + Cloak	Che từ cổ trở lên	Trung bình
`obliterate`	4-step pipeline	Compress → Scramble → Blur → Quantize	Chậm
`silhouette`	Extreme cloak	Randomize tone + noise + GBlur + Quantize	Chậm nhất

Listing 13: src/privacy.py — obliterate (mạnh nhất)

```
def _obliterate_features(roi, obliterate_scale, ...):
    """4-buoc pha huy thong tin nhan dang - KHONG the khoi phuc"""

    # Buoc 1: Compress xuong 8% -> xoa het chi tiet mat
    work = cv2.resize(roi, (w*0.08, h*0.08), INTER_AREA)
    work = cv2.resize(work, (w, h), INTER_NEAREST)

    # Buoc 2: Xao tron block 12x12 -> pha vo quan he hinh hoc
    work = _scramble_blocks(work, block_size=12, seed=1337)

    # Buoc 3: Gaussian Blur 35x35 -> xoa canh con lai
```

```
    work = cv2.GaussianBlur(work, (35, 35), 0)

    # Buoc 4: Quantize mau ve 10 muc -> xoa texture tinh te
    work = _quantize_colors(work, levels=10)
    return work
```

5.6 Demo Decrypt trên Secure Archive

24:30 - 25:00

THAO TÁC

Mở browser, chuyển sang tab `http://localhost:8000/admin/secure-archive.html`.

[ĐẠT] Demo decrypt

"Đây là Secure Archive — trang chỉ admin truy cập. Danh sách các clip `.enc` đã được lưu. Em sẽ decrypt một clip:"

THAO TÁC

Thao tác:

1. Click vào một clip trong danh sách
2. Click **Decrypt & Play**
3. Nhập admin password vào modal
4. Clip gốc (chưa blur) hiển thị

[ĐẠT] Giải thích decrypt

"Và đây là clip gốc — khuôn mặt hoàn toàn rõ nét.
Nếu nhập sai password, server trả về lỗi: *'Decryption failed – wrong password or corrupted clip'*— không leak bất kỳ thông tin nào về độ gần đúng của password.
Lưu ý: decrypt xảy ra *server-side*. File `.enc` không bao giờ gửi xuống browser. Chỉ video đã decode được stream về — thêm một lớp bảo vệ."

6. Phần 5 — Demo End-to-End + Q&A

25:00 - 30:00

6.1 Demo end-to-end hoàn chỉnh

25:00 - 27:30

[ĐẠT] Giới thiệu demo

"Bây giờ chúng em sẽ demo toàn bộ flow từ đầu đến cuối."

Bước 1: Reset và start pipeline

THAO TÁC

Khánh thao tác: stop pipeline nếu đang chạy, start lại với mode **obliterate**.

[KHÁNH]

"Pipeline đã start với chế độ obliterate — mạnh nhất."

Bước 2: Thành viên đã đăng ký

THAO TÁC

Khoa đứng trước camera.

[ĐẠT]

"Khoa đã được đăng ký trong database. Hệ thống nhận ra Khoa với similarity cao — khuôn mặt **không bị blur**, và tên hiển thị bên cạnh."

THAO TÁC

Đạt đứng trước camera.

[ĐẠT]

"Đạt cũng đã đăng ký — nhận ra, không blur."

Bước 3: Người lạ

THAO TÁC

Khánh (chưa đăng ký) đứng trước camera.

[ĐẠT]

"Khánh chưa đăng ký vào database. Hệ thống không nhận ra → khuôn mặt bị **obliterate hoàn toàn**. Không ai xem stream này có thể biết Khánh trông như thế nào."

Bước 4: Mở Secure Archive

THAO TÁC

Đạt mở `http://localhost:8000/admin/secure-archive.html`, decrypt clip vừa ghi.

[ĐẠT]

"Clip vừa ghi đã tự động lưu vào `data/clips/secure/`. Sau khi decrypt, ta thấy khuôn mặt đầy đủ của Khánh trong clip gốc.
Đây chính là mục tiêu: Khánh được bảo vệ quyền riêng tư khi xem video công khai, nhưng bằng chứng vẫn được bảo toàn hoàn chỉnh cho admin."

6.2 Privacy Modes Showcase

27:30 - 29:00

THAO TÁC

Đạt lần lượt switch qua từng mode trên dashboard, chỉ cho thầy cô thấy kết quả trực tiếp trên stream.

Demo sequence

1. `blur` — *"Cơ bản, vẫn còn thấy outline khuôn mặt mờ mờ"*
2. `pixelate` — *"Mosaic, style camera truyền hình. Nhanh hơn blur"*
3. `headcloak` (default) — *"Che toàn đầu + noise. Mặc định của hệ thống"*
4. `obliterate` — *"4-bước pipeline, không còn trace nào. Dùng cho bảo vệ cao nhất"*
5. `silhouette` — *"Cực mạnh: che đầu và vai, quantize màu còn 6 mức"*

6.3 Kết luận

29:00 - 30:00

[ĐẠT] Kết luận

"Tóm lại, hệ thống chúng em đạt được **3 mục tiêu chính**:

1. Privacy bởi vì mặc định (Privacy by Default): Bất kỳ người lạ nào đi vào vùng quan sát đều bị ẩn danh hoá ngay lập tức, không cần can thiệp thủ công.

2. Security đúng nghĩa: AES-256-GCM + PBKDF2-SHA256 390.000 iterations + constant-time comparison. Không có backdoor, không có plaintext password, không có timing leak.

3. Usability thực tế: REST API đầy đủ cho frontend integration, hot-reload config không cần restart, 7 chế độ blur để phù hợp nhiều use case.

Nhóm chúng em xin cảm ơn thầy cô và **sẵn sàng trả lời câu hỏi**."

7. Câu Hỏi Thường Gặp — Đáp Án Chuẩn Bị Sẵn

Q1: Tại sao dùng AES-GCM mà không phải AES-CBC?

Trả lời (Đạt): AES-GCM là *Authenticated Encryption with Associated Data (AEAD)*. Ngoài mã hóa dữ liệu, GCM còn tạo ra một **Authentication Tag 128-bit**. Khi decrypt, nếu ciphertext bị sửa đổi dù chỉ 1 bit, authentication sẽ fail.
AES-CBC chỉ mã hóa — không đảm bảo toàn vẹn. Kẻ tấn công có thể chỉnh sửa file `.enc` và nhận lại dữ liệu sai mà không biết.

Với bằng chứng pháp lý, **integrity là bắt buộc, không phải tùy chọn**.

Q2: Nếu admin quên mật khẩu thì sao?

Trả lời (Đạt): Không khôi phục được — đây là *design có chủ ý*.
Hệ thống không lưu plaintext, không có secret recovery key, không có backdoor. Đây là nguyên tắc **Zero Knowledge of Plaintext**.
Trong scope đồ án, ta chọn *security over convenience*. Trong production thực tế, có thể áp dụng thêm cơ chế **Key Escrow** hoặc **Shamir Secret Sharing** để chia key giữa nhiều người được ủy quyền.

Q3: Tại sao 390.000 iterations, không phải nhiều hơn?

Trả lời (Đạt): Con số này là khuyến nghị của **OWASP 2023** cho PBKDF2-SHA256 trên phần cứng phổ thông năm 2023.
Đây là *balance*: đủ chậm để brute-force không khả thi, nhưng đủ nhanh để user thực tế không cảm thấy lag khi encrypt/decrypt một clip (xảy ra rất ít, không phải mỗi frame).
Nếu CPU server mạnh hơn, có thể tăng lên 600.000+ iterations theo khuyến nghị mới hơn.

Q4: Real-time có bị lag không? FPS bao nhiêu?

Trả lời (Khánh): YOLO detect mỗi 2 frame (`detect_every=2`). Process width 640px với preset `balanced`. Trên CPU mid-range đạt khoảng **15–20 FPS** — đủ cho camera giám sát.
Nếu cần nhanh hơn: Chuyển preset sang `fast`: YOLO size 416, pixelate, detect mỗi 3 frame, có thể đạt 25–30 FPS.
Nếu có GPU, thay `CPUExecutionProvider` bằng `CUDAExecutionProvider` cho InsightFace, tốc độ nhận diện bào nhận diện tăng gấp 5–10 lần.

Q5: Nhận diện nhầm thì sao? Privacy leakage?

Trả lời (Khoa): False negative (nhận diện sai thành viên là người lạ) → thành viên bị blur nhưng clip gốc vẫn có. Không ảnh hưởng security.
False positive (người lạ nhầm thành thành viên) → người lạ không bị blur trong stream. Đây là rủi ro privacy.
Để giảm thiểu, ngưỡng threshold được calibrate cẩn thận với 100 ảnh mẫu/người. Threshold 0.35 được chọn sau benchmarking để minimize false positive ở mức chấp nhận được.

Q6: GDPR / Luật bảo vệ dữ liệu?

Trả lời (Đạt): Hệ thống thiết kế theo nguyên tắc **Privacy by Design** (GDPR Article 25):

- Dữ liệu biometric (embedding) chỉ lưu trong mạng nội bộ
- Clip gốc mã hóa, giới hạn truy cập
- Có thể delete clip và member bất cứ lúc nào

- Không gửi ảnh khuôn mặt ra bên ngoài server

Trong production cần bổ sung: audit log, consent management, data retention policy.

8. Checklist Kỹ Thuật Trước Buổi Trình Bày

Checklist 30 phút trước

- ☐ Server đang chạy trên port 8000
- ☐ Admin credentials tồn tại: `Test-Path data/admin_credentials.json`
- ☐ Có ít nhất 2 clip `.enc` trong `data/clips/secure/`
- ☐ Camera nhận diện được: `GET http://localhost:8000/cameras`
- ☐ Thành viên Đạt và Khoa đã trong database
- ☐ VS Code: font size $\geq$ 16, file explorer bên trái đóng bớt folder dư
- ☐ Browser: 3 tab mở sẵn, zoom 110%
- ☐ Tắt notification Windows (Focus Assist: Alarms only)
- ☐ Tắt screensaver và sleep
- ☐ Backup: có sẵn clip `.enc` test nếu camera lỗi live

Tình huống khẩn cấp

Camera không hoạt động: Dùng clip demo quay sẵn. Giải thích: "Do giới hạn phòng thi, em demo trước."
Server crash: `uvicorn src.app_api:app -port 8000` — khởi động lại $<$ 10 giây.
Quên từ: Không sao — dừng, nhìn vào file code, giải thích từ code. Code là "notes" của lập trình viên.
Câu hỏi không biết: "Em chưa nghiên cứu sâu phần này, nhưng theo kiến thức của em là... Thầy/cô có thể cho em bổ sung thêm không?"

— Chúc nhóm trình bày thành công! —